

Assignment 2

Full Name:	Ajay Mahadev Avale
DIV:	IT_SEDA
Roll No.:	22
PRN:	1252130022
Subject:	Automata Theory IT2308
Date of Conduction:	
Date of Submission:	

Assignment No: 2

Subject : Automata Theory / compiler Design

Topic : Lexical Analysis using Finite Automata.

- Objective of Assignment :

To study how finite automata can be used to recognize basic lexical elements such as identifiers, keywords, operators and relational operators.

- * Question 1:

Design a Finite Automata to recognize an identifier.

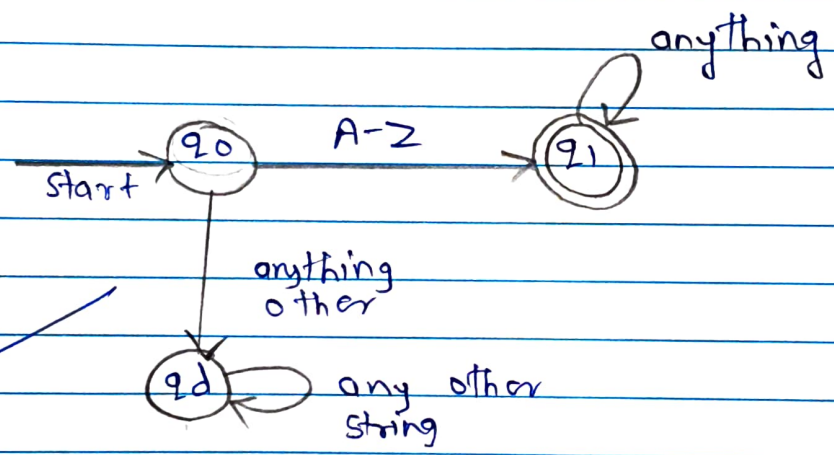
- Aim :

To design a finite automata that checks whether a given string is a valid identifier.

Theory:

An identification of identifier must start with a letter or underscore and may contain letters, digits or underscores.

DFA diagram:



Input	String
A-Z	Accept
a-z	Accept
_	Accept
other	Reject

- Simulation:

consider a string "abi"

$\delta(q_0, -)$



$\delta(q_1, a)$



$\delta(q_1, b)$



$\delta(q_1, i) \Rightarrow q_1 \text{ (final state)}$

String ends at q_1 (accepting state) hence string is accepted.

- Conclusion:

The automation accepts valid identifier and retrieves invalid.

- * Question 2

Keyword Recognition

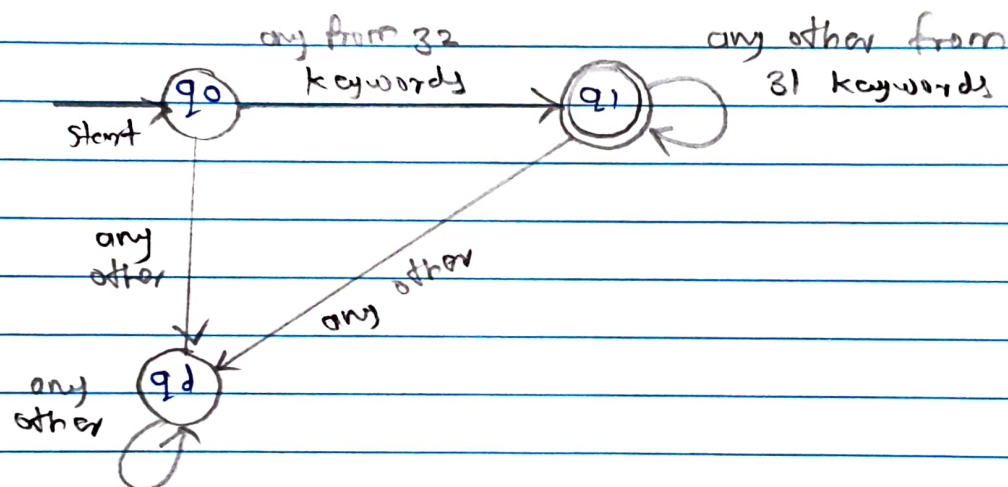
- * Aim:

To design a finite automaton to recognize keywords.

• Theory :

keywords are reserved words with predefined meaning in a programming language.

• DFA diagram :



Input	Accept
Any keyword from 32 keywords	Accept
other	Reject

- Simulation :

Considering a string as for :

$$\delta(q_0, \text{for}) = q_1 (\text{final state})$$

Hence String accepted.

- * Question 3:

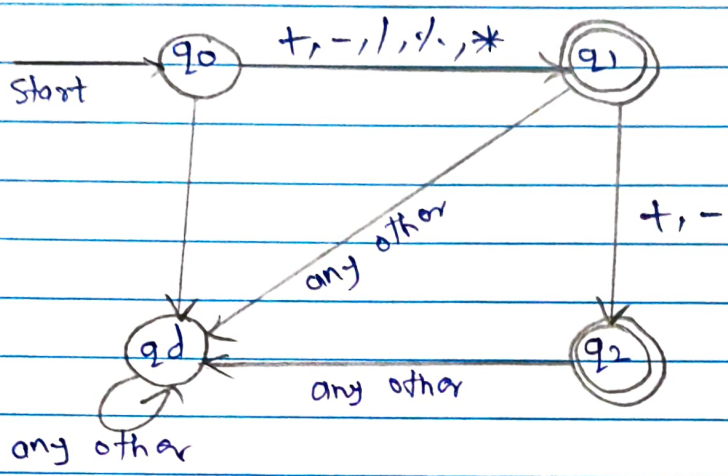
Operator Recognition :

- Aim :

To design a finite automation to recognize operators.

- Theory :

Operators are symbols used to perform operations such as +, -, *, /.



Input	State
+ , - , / , % , * , + , -	Accept
Any other	Reject

• Simulation :

Consider string '+'.

$$\delta(q_0, +) = q_1 \text{ (acceptance state)}$$

• Conclusion :

The automaton accepts valid operators.

Code :

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>


int main()
{
    char s[20];
    int i, valid, choice;
    int ans = 1;


    do
    {
        valid = 1;


        printf("\nEnter a string: ");
        scanf("%s", s);


        printf("\nSelect option:\n");
        printf("1. Check Operator\n");
        printf("2. Check Keyword\n");
        printf("3. Check Identifier\n");


        printf("Enter your choice: ");
        scanf("%d", &choice);


        switch (choice)
        {
            case 1:
                if (strlen(s) == 1 &&
```



```
(s[0] == '+' || s[0] == '-' || s[0] == '*' ||  
s[0] == '/' || s[0] == '=')  
printf("It is an Operator\n");  
else  
    printf("It is NOT an Operator\n");  
break;
```

case 2:

```
if (strcmp(s, "int") == 0 || strcmp(s, "if") == 0 ||  
    strcmp(s, "else") == 0 || strcmp(s, "while") == 0)  
    printf("It is a Keyword\n");  
else  
    printf("It is NOT a Keyword\n");  
break;
```

case 3:

```
if (isalpha(s[0]))  
{  
    for (i = 1; s[i] != '\0'; i++)  
    {  
        if (!isalnum(s[i]))  
        {  
            valid = 0;  
            break;  
        }  
    }  
}  
  
if (valid)  
    printf("It is an Identifier\n");  
else
```

```
        printf("Invalid string\n");
    }
    else
        printf("Invalid string\n");
    break;

default:
    printf("Invalid choice\n");
}

printf("\nDo you want to continue? (1 = Yes / 0 = No): ");
scanf("%d", &ans);

} while (ans == 1);

return 0;
}

;
```

Output:

```
Enter a string: +

Select option:
1. Check Operator
2. Check Keyword
3. Check Identifier
Enter your choice: 1
It is an Operator

Do you want to continue? (1 = Yes / 0 = No): 1

Enter a string: int

Select option:
1. Check Operator
2. Check Keyword
3. Check Identifier
Enter your choice: 2
It is a Keyword

Do you want to continue? (1 = Yes / 0 = No): 1

Enter a string: abc123

Select option:
1. Check Operator
2. Check Keyword
3. Check Identifier
Enter your choice: 3
It is an Identifier

Do you want to continue? (1 = Yes / 0 = No): |
```